

Practice Set: C++ Program Structure & I/O Streams

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Explain the exact role of the preprocessor directive `#include <iostream>` in a C++ program. Why is the legacy `<iostream.h>` syntax deprecated in modern standard C++?

Question 2: What is a **namespace** in C++? Explain why using namespace `std`; is used and why relying on it globally in header files is considered bad practice in professional C++ development.

Question 3: Describe the significance of the `main()` function's `int` return type. What do returning 0 and returning a non-zero value (e.g., 1 or -1) signal to the Operating System?

Question 4: Define a **Stream** in C++. Draw/describe the data flow in memory between physical input/output hardware and RAM for standard input and output operations.

Question 5: Name the base classes responsible for input and output operations in C++ and state which derived class inherits from both to provide bidirectional I/O capabilities.

Question 6: Differentiate between the stream objects `cout`, `cin`, `cerr`, and `clog` in terms of their standard stream target, class type, and buffering behavior.

Question 7: Define the **Insertion Operator** (`<<`) and the **Extraction Operator** (`>>`). Detail their data flow directions and formal names.

Question 8: Compare `'\n'` and `endl` in detail. Explain why using `endl` excessively inside heavy loop constructs causes performance degradation.

Question 9: What are **Tokens** in C++? List the five distinct categories of language tokens identified during lexical analysis.

Question 10: Write a complete C++ program that prompts the user to enter their age, reads the input from the keyboard, and prints back a response using stream operators and `endl`.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): What is the main structural difference between `cerr` and `clog` if both send diagnostic messages to the error screen?

Question 12 (Viva): Which C++ stream object is associated with line-buffered input from the keyboard?

Question 13 (Viva): What operation does `std::flush` perform, and which manipulator automatically triggers it?

Question 14 (Viva): What happens during the compilation pipeline when the preprocessor encounters a line starting with `#`?

Question 15 (Viva): How do you explicitly scope stream objects like `cout` or `cin` without including the statement using namespace `std`;

Question 16 (Viva): What is the informal name given to the Stream Insertion Operator (`<<`)?

Question 17 (Viva): What is the informal name given to the Stream Extraction Operator (`>>`)?

Question 18 (Viva): Name two C++ keywords used in the standard program boilerplate shown in the notes.

Question 19 (Viva): Why does modern C++ require the return type of `main()` to be `int` rather than `void`?

Question 20 (Viva): Is a C++ string literal like "Hello World" considered an identifier, a variable, or a constant token?

Answers & Explanations

Answer: 01

`#include <iostream>` tells the preprocessor to copy the declarations of standard input and output stream classes (like `istream` and `ostream`) into the file before compilation.

Modern standards (C++98 and beyond) deprecated `<iostream.h>` to standardize the C++ Standard Library headers without `.h` extensions and enclose them within the `std` namespace.

Answer: 02

A namespace is a declarative region that provides a scope to identifiers (functions, classes, variables) to prevent naming collisions.

using namespace std; allows direct access to standard library entities like cout and cin without explicit scoping.

Best Practice Warning: Using it globally in header files introduces naming conflicts across files that include that header. Modern C++ prefers explicit scoping (e.g., std::cout).

Answer: 03

The int return type sends an execution exit status code back to the Operating System.

Returning 0 signals successful execution without runtime errors.

Returning a non-zero value (e.g., 1 or -1) signals an abnormal termination or runtime error.

Answer: 04

A **Stream** is an abstraction representing a continuous sequence of bytes flowing between physical hardware devices (keyboard, monitor, disk) and computer RAM memory.

Input Flow: Keyboard -----> istream/cin Buffer ----> RAM Memory Variable.

Output Flow: RAM Variable/Literal -----> ostream/cout Buffer ----> Display Screen.

Answer: 05

istream: Base class designed for reading input sequential bytes.

ostream: Base class designed for writing output sequential bytes.

iostream: Derived class inheriting from both istream and ostream to provide bidirectional stream operations.

Answer: 06

Stream Object	Class Type	Target Device	Buffering Behavior
cin	istream	Standard Input (Keyboard)	Line-buffered
cout	ostream	Standard Output (Console Screen)	Buffered
cerr	ostream	Standard Error Screen	Unbuffered (Flushes errors immediately)
clog	ostream	Standard Log Device (Screen/File)	Buffered diagnostic logging

Answer: 07

Insertion Operator (<<): Formally called the **Put-To Operator**. Inserts data from its right-hand operand into the output stream specified on its left (e.g., cout << data).

Extraction Operator (>>): Formally called the **Get-From Operator**. Extracts data from the input stream specified on its left and assigns it to the variable specified on its right (e.g., cin >> variable).

Answer: 08

'\n' is an escape character that inserts a newline character into the output stream buffer **without** forcing an immediate buffer flush (fast & optimal).

endl is a stream manipulator that inserts a newline character **and** explicitly flushes the stream buffer via std::flush.

Performance Issue: Overusing endl in heavy loops forces repeated physical screen/disk flush operations, causing significant execution delays.

Answer: 09 Tokens are the smallest individual building blocks recognized by the compiler during lexical analysis. The five core categories are:

Keywords (e.g., int, return)

Identifiers (e.g., main, userAge)

Variables (Memory Labels)

Constants (e.g., 10, 'A', 3.14)

Operators (e.g., <<, >>)

Answer: 10

```
C++
#include <iostream>
using namespace std;
int main()
{
    int userAge;
    cout << "Enter your age: ";
    cin >> userAge;
    cout << "You entered: " << userAge << " years old." << endl;
    return 0;
}
```

Answer: 11 `cerr` is **unbuffered** and flushes error messages to the screen immediately, whereas `clog` is **buffered** and stores log messages in memory before writing

Answer: 12 `cin` (an instance of the `istream` class)

Answer: 13 `std::flush` explicitly flushes all pending buffered bytes to the physical output hardware. It is automatically triggered by the `endl` manipulator.

Answer: 14 Lines starting with `#` are handled before compilation starts; the preprocessor directly includes header code or resolves macro directives into the source file.

Answer: 15 By using the scope resolution operator `::` directly attached to the namespace, such as `std::cout` and `std::cin`

Answer: 16 The **Put-To Operator**

Answer: 17 The **Get-From Operator**.

Answer: 18 `using` and `return` (or `int`)

Answer: 19 Operating systems require an integer return code from ``main()`` to determine whether the program executed successfully (``0``) or failed with an error status (non-zero)

Answer: 20 It is classified as a **Constant** token.